プログラミングワークショップ 2

青森大学ソフトウェア情報学部

橋本恭能

haship@aomori-u.ac.jp

講義資料について

- この講義で使う資料やプリントは以下のWebページから閲覧できます
- ブラウザを開き、以下のアドレスを入力してください

URL http://haship.github.io

- Webページ上部の【講義資料】をクリック
- 表から【プログラミングワークショップ２】をクリック
- 目次ページをブックマークする

目次

- テーマ：ボールとブロックの当たり判定
- 例題：Blockクラス追加
- 課題：ボールが衝突したらブロックを消す

プログラム作成

- 課題05にブロッククラスを追加
  - フィールド
    - x座標、y座標、幅、高さ
    - 衝突したかどうか（ブロックが見えているかどうか）
  - メソッド
    - ボールと(左・右・上・下)側で衝突したかどうか
    - ボールと頂点で衝突したかどうか
    - 消す
    - 描く

Blockクラス

- 新規タブ Block
- Barクラスをコピーして修正
  - 追加
    - フィールド　boolean visible;
    - メソッド　void remove()
  - 削除
    - フィールド　float speed;
    - メソッド　void move()

ブロックは移動しないので
speed, move()を削除

Blockクラス　Barクラスをコピーして修正

```
class Block {

  float x;
  float y;
  float w;
  float h;
  boolean visible;

  Block(float tempX, float tempY) {
    x = tempX;
    y = tempY;
    w = 80;
    h = 20;
    visible = true;
  }

  void display(){
    rect(x, y, w, h);
  }

  void remove(){
    visible = false;
  }
```

追加

追加

追加

```
  boolean isHitCorner(Ball b) {
    if(dist(b.x, b.y, x, y) < b.radius ||
     dist(b.x, b.y, x, y+h) < b.radius ||
     dist(b.x, b.y, x+w, y) < b.radius ||
     dist(b.x, b.y, x+w, y+h) < b.radius) {
      return true;
    } else {
      return false;
    }
  }
  boolean isHitTopSide(Ball b) {
    if (b.x > x && b.x < x+w &&
     dist(b.x, b.y, b.x, y) < b.radius) {
      return true;
    } else {
      return false;
    }
  }
  boolean isHitBottomSide(Ball b) {
    if (b.x > x && b.x < x+w &&
     dist(b.x, b.y, b.x, y+h) < b.radius) {
      return true;
    } else {
      return false;
    }
  }
```

Blockクラス Barクラスをコピーして修正

```
 boolean isHitLeftSide(Ball b) {
    if (b.y > y && b.y < y+h &&
     dist(b.x, b.y, x, b.y) < b.radius) {
      return true;
    } else {
      return false;
    }
  }

  boolean isHitRightSide(Ball b) {
    if (b.y > y && b.y < y+h &&
     dist(b.x, b.y, x+w, b.y) < b.radius) {
      return true;
    } else {
      return false;
    }
  }

}

// フィールド speed 削除
// メソッド move() 削除
```

メインタブ

```
Ball ball;
Bar bar;
Block block;

void setup() {
  size(400, 400);
  ellipseMode(RADIUS);
  ball = new Ball(110, 60, 0.5, 0.3);
  bar = new Bar(width/2, height*0.8);
  block = new Block(40, height*0.2);
}
void draw() {
  background(0);

  ball.move();
  bar.move();

  if (ball.isHitRightWall() || ball.isHitLeftWall()) {
    ball.flipXdir();
  }
  if (ball.isHitBottomWall() || ball.isHitTopWall()) {
    ball.flipYdir();
  }
```

追加

追加

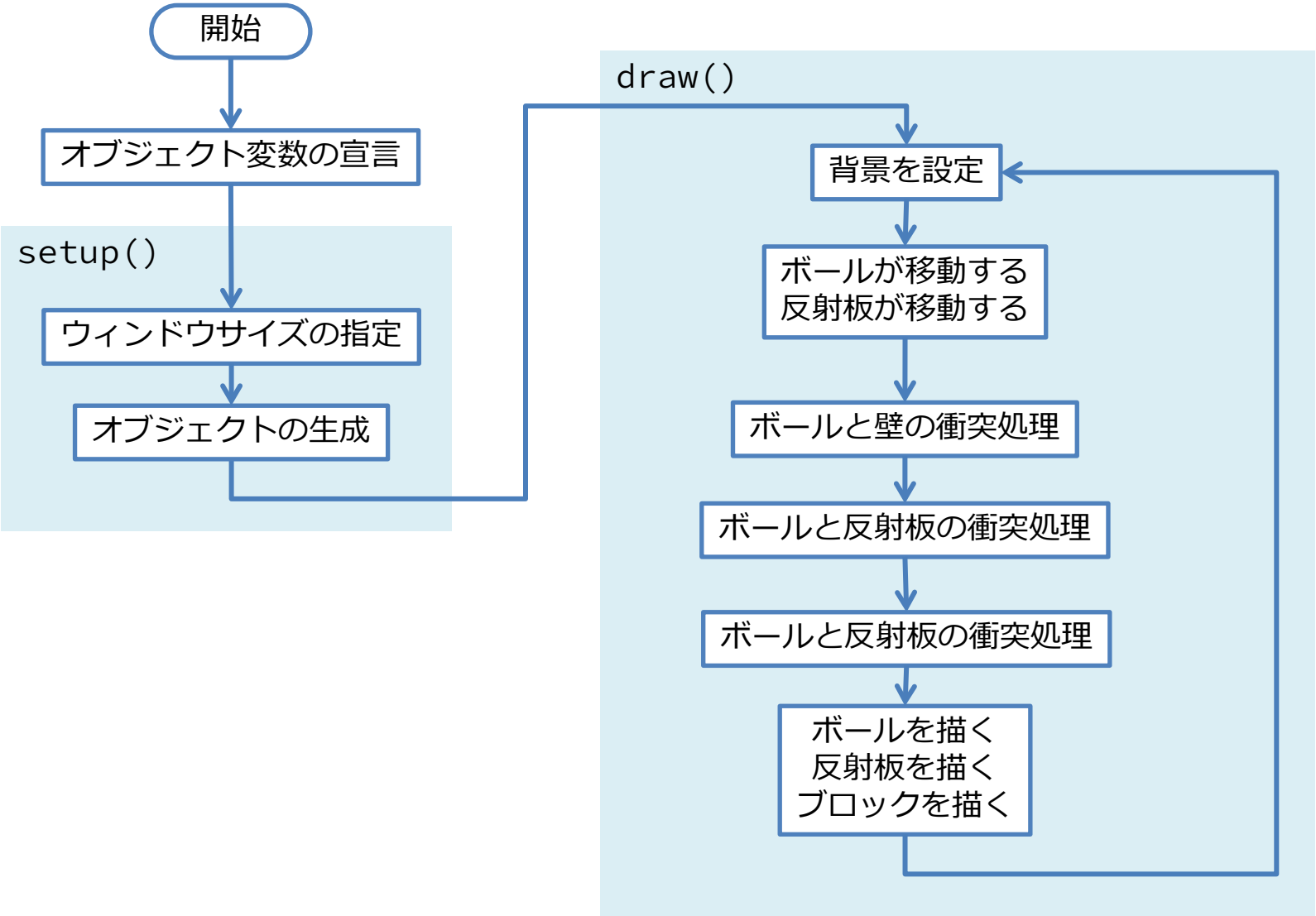
メインタブ

```
  if(bar.isHitTopSide(ball) || bar.isHitBottomSide(ball)){
    ball.flipYdir();
  }
  if(bar.isHitRightSide(ball) || bar.isHitLeftSide(ball)){
    ball.flipXdir();
  }
  if(bar.isHitCorner(ball)){
    ball.flipXdir();
    ball.flipYdir();
  }

  if(block.isHitTopSide(ball) || block.isHitBottomSide(ball)){
    ball.flipYdir();
  }
  if(block.isHitRightSide(ball) || block.isHitLeftSide(ball)){
    ball.flipXdir();
  }
  if(block.isHitCorner(ball)){
    ball.flipXdir();
    ball.flipYdir();
  }

  ball.display();
  bar.display();
  block.display();
}
```

追加

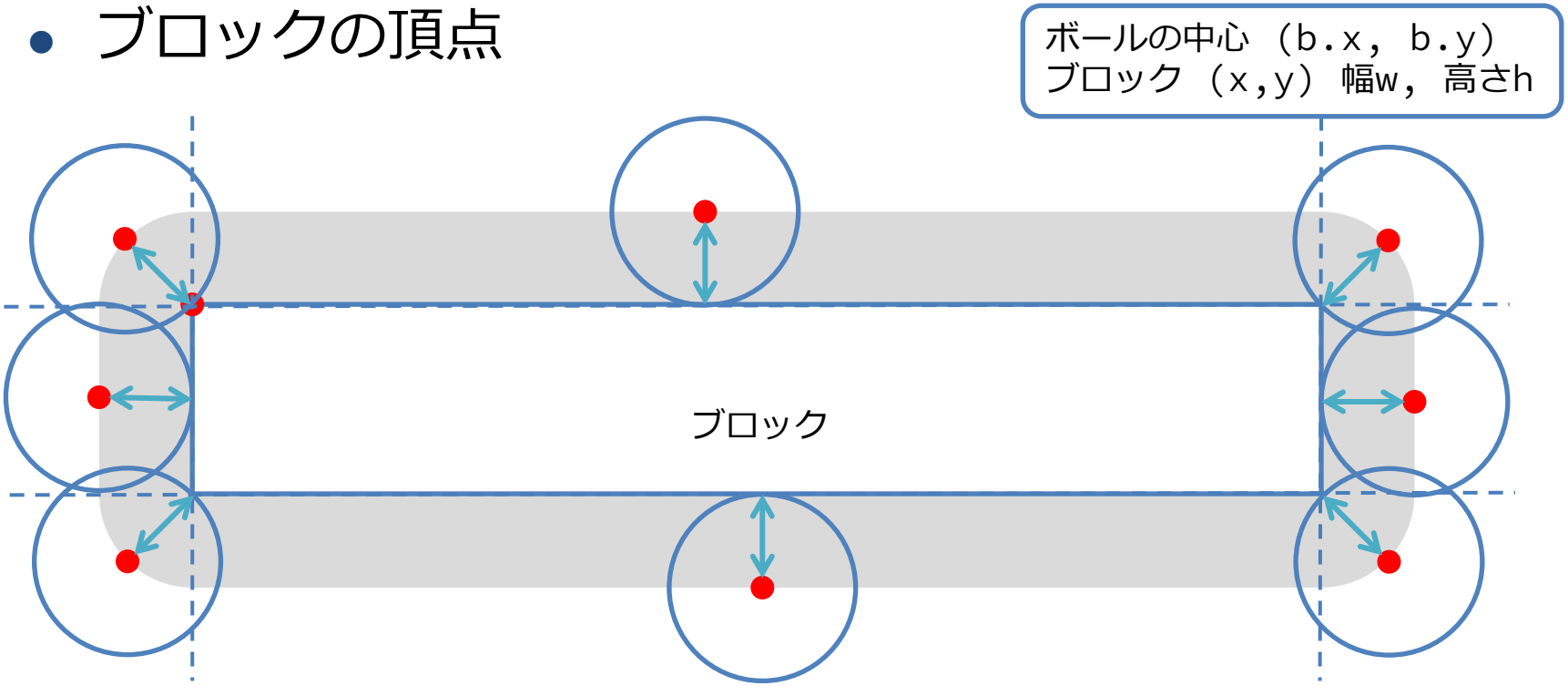
追加

フローチャート③　大まかな処理



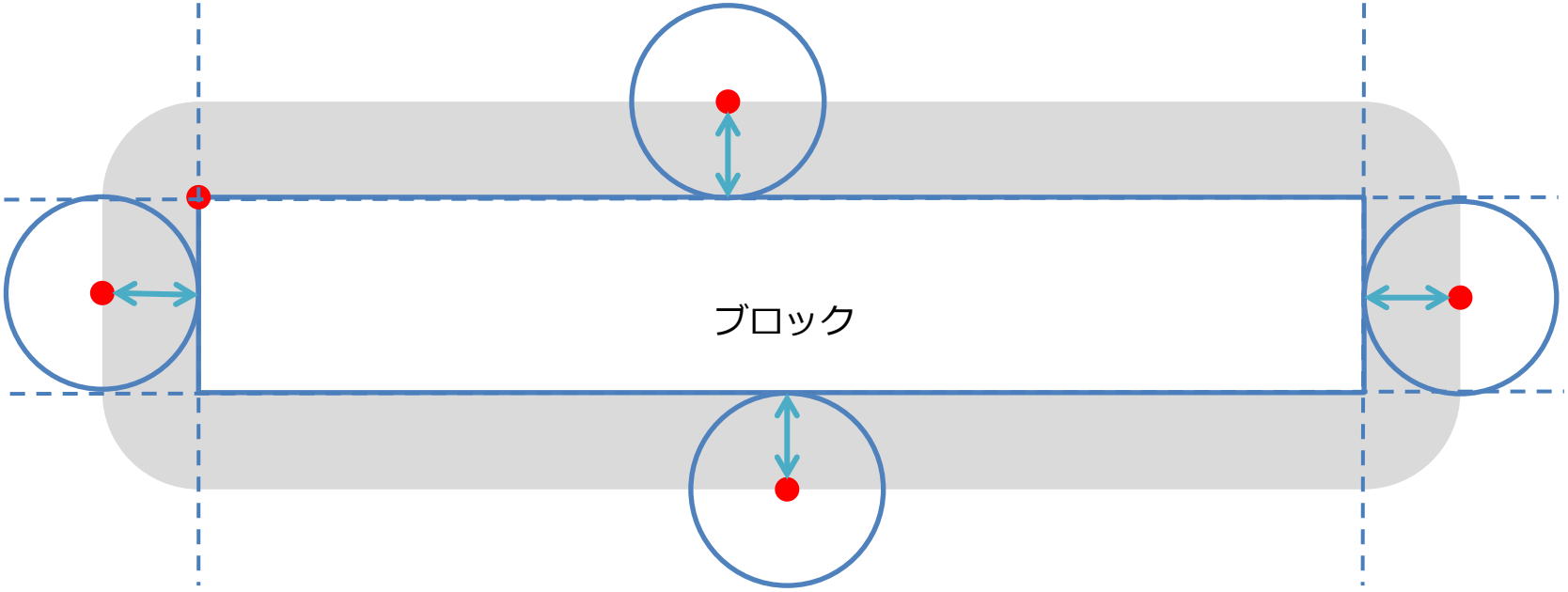
ボールとブロックの衝突

- ボールと反射板の衝突と基本同じ
- ボールとブロックが衝突するところ
  - ブロックの辺(上側、下側、左側、右側)
  - ブロックの頂点



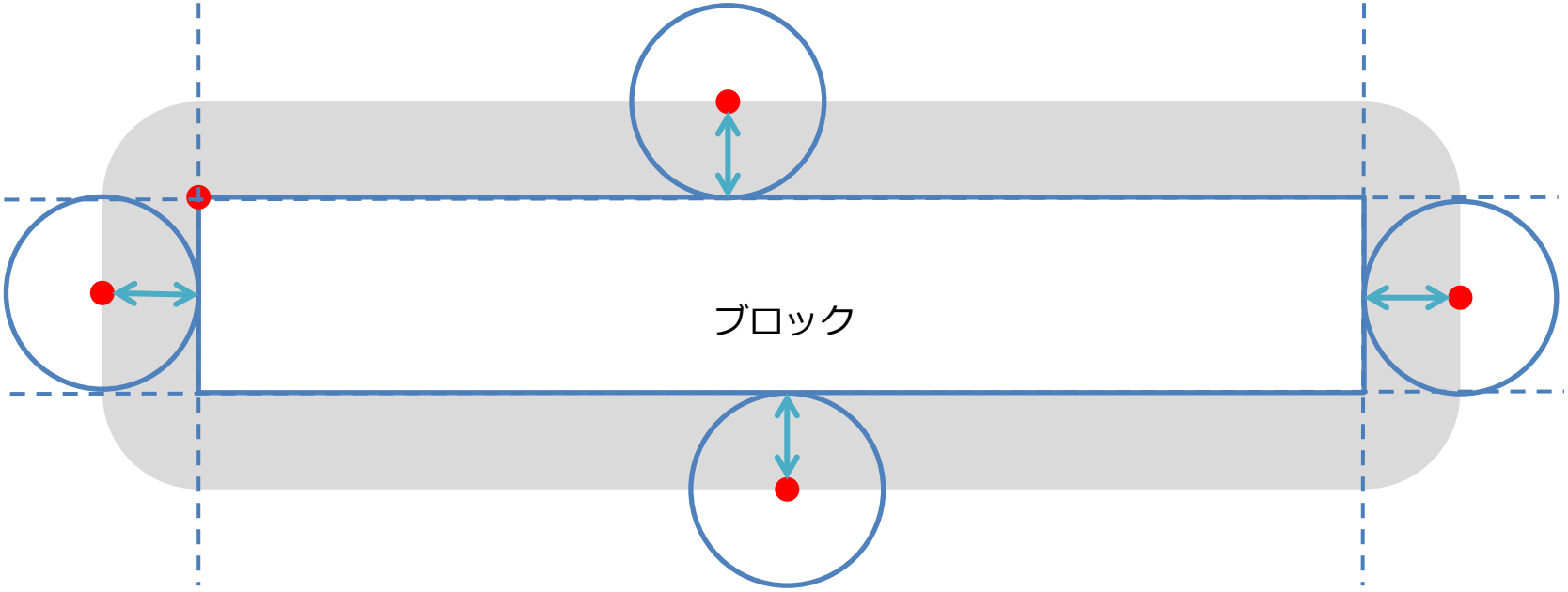
ボールとブロックの衝突（辺）

- ブロックの上下左右にボールの半径分だけ拡大した範囲（灰色の部分）にボールの中心があるかどうか



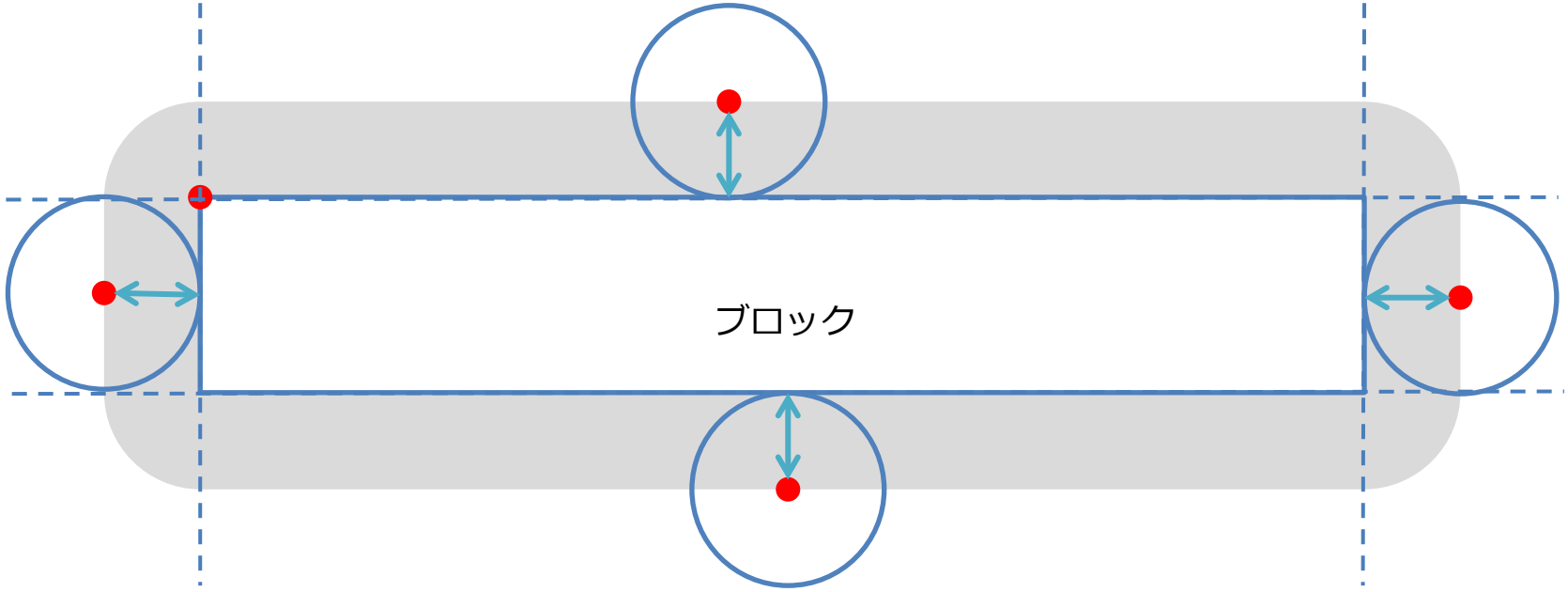
ボールとブロックの衝突（上側）

- ブロック上側
  - ボールのxがブロックのxより大きい　　`b.x > x`
  - ボールのxがブロックのx+wより小さい　　`b.x < x+w`
  - ボールとブロック上側の距離がボールの半径より小さい
    `dist(b.x, b.y, b.x, y) < b.radius`



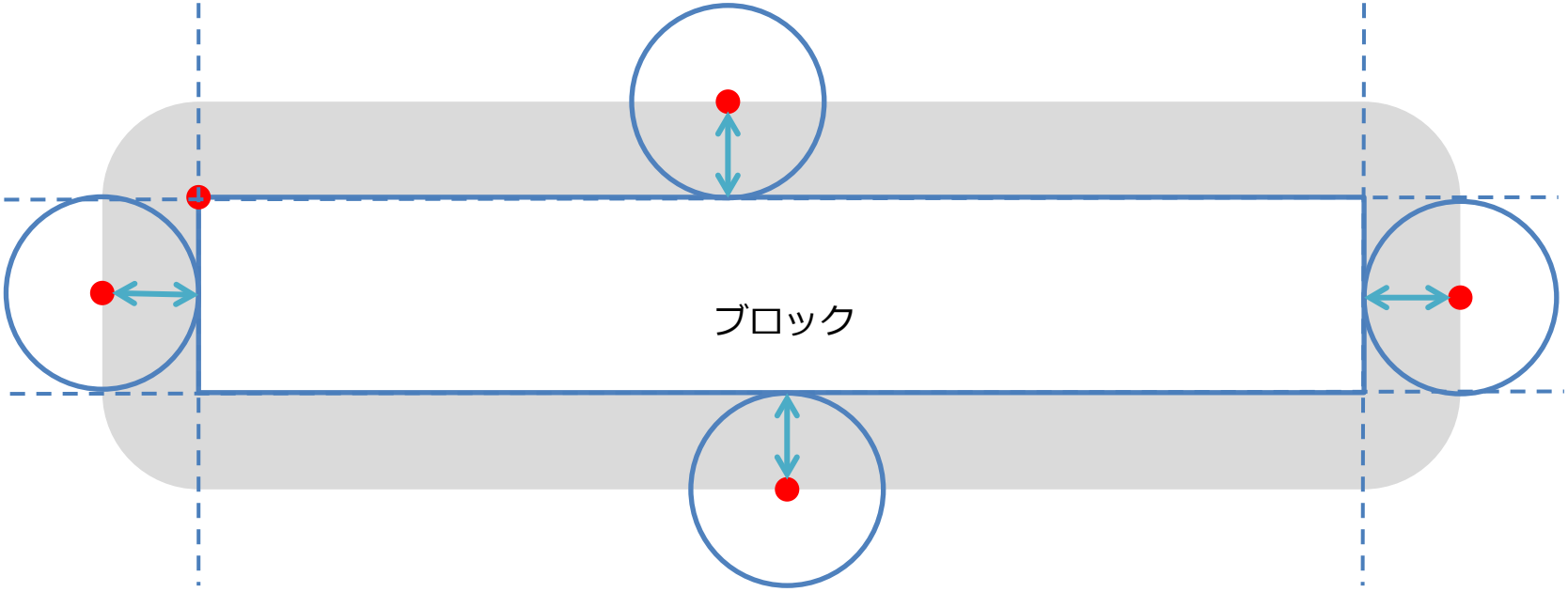
ボールとブロックの衝突（下側）

- ブロック下側
  - ボールのxがブロックのxより大きい　　`b.x > x`
  - ボールのxがブロックのx+wより小さい　　`b.x < x+w`
  - ボールとブロック下側(b.x,y+h)の距離がボール半径より小さい
    `dist(b.x, b.y, b.x, y+h) < b.radius`



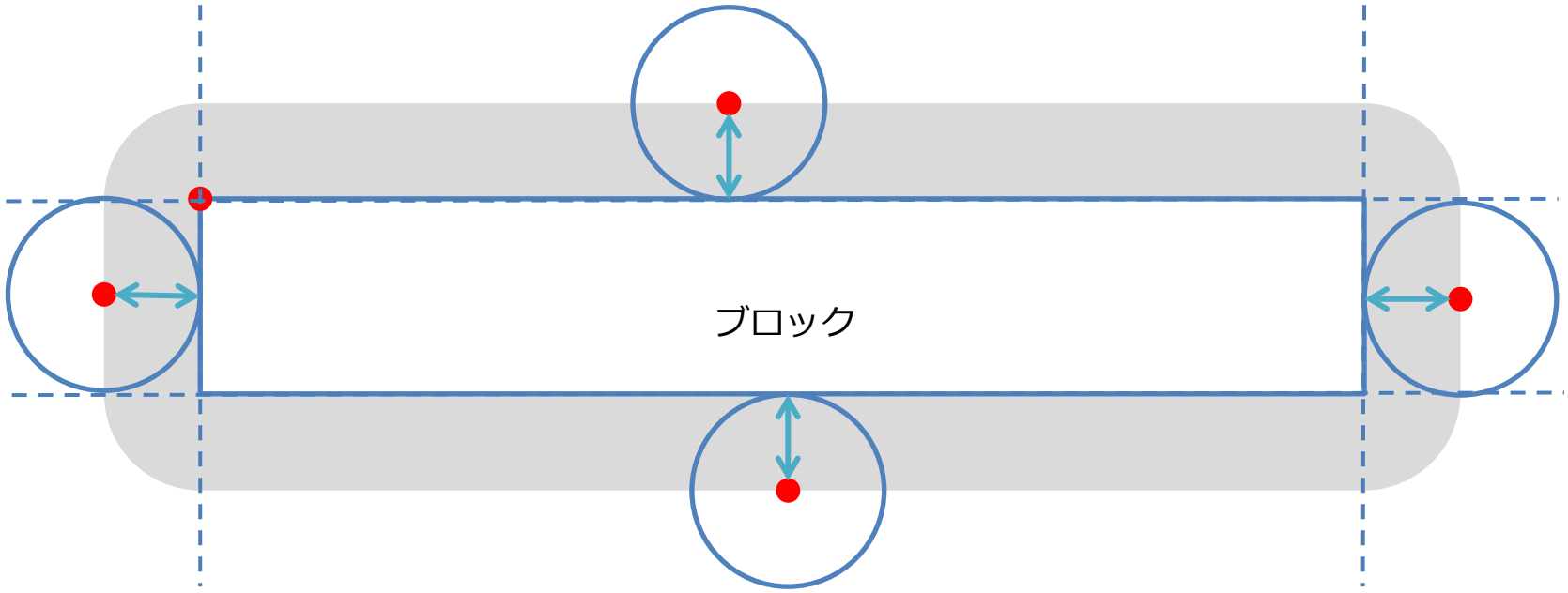
ボールとブロックの衝突（左側）

- ブロック左側
  - ボールのyがブロックのyより大きい　　`b.y > y`
  - ボールのyがブロックのy+hより小さい　　`b.y < y+h`
  - ボールとブロック左側の距離がボール半径より小さい
    `dist(b.x, b.y, x, b.y) < b.radius`



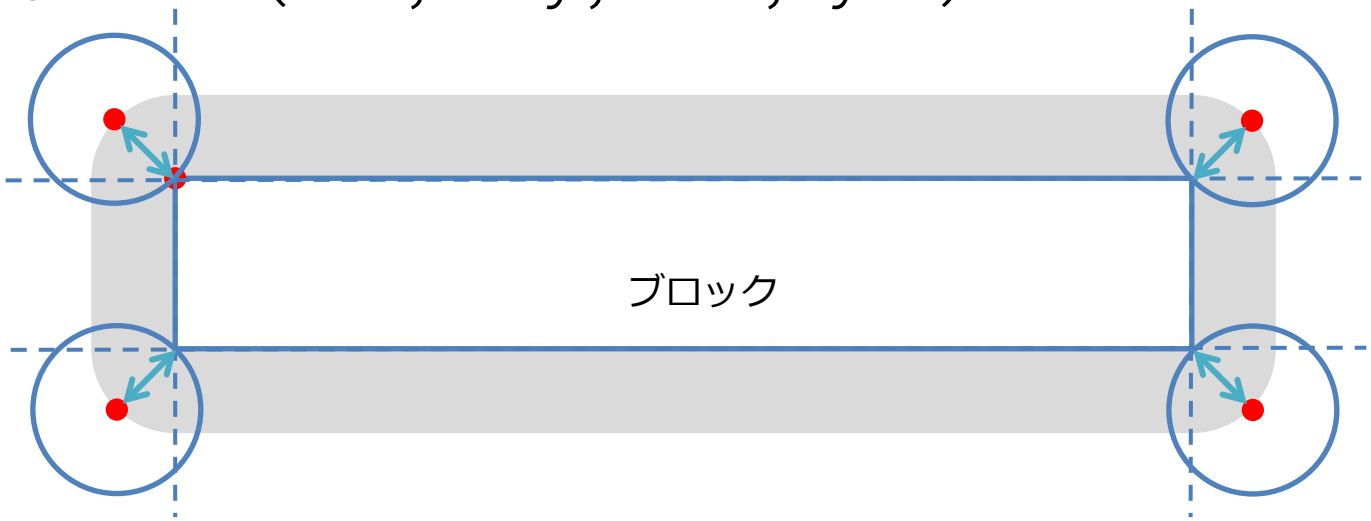
ボールとブロックの衝突（右側）

- ブロック右側
  - ボールのyがブロックのyより大きい　　`b.y > y`
  - ボールのyがブロックのy+hより小さい　　`b.y < y+h`
  - ボールとブロック右側の距離がボール半径より小さい
    `dist(b.x, b.y, x+w, b.y) < b.radius`



ボールとブロックの衝突（頂点）

- ブロックの頂点とボールの中心の距離がボールの半径より小さい
  - 左上：`dist(b.x, b.y, x, y) < b.radius`
  - 左下：`dist(b.x, b.y, x, y+h) < b.radius`
  - 右上：`dist(b.x, b.y, x+w, y) < b.radius`
  - 右下：`dist(b.x, b.y, x+w, y+h) < b.radius`



ボールとブロックの衝突

- 衝突した場合の処理
  - ブロック上側、下側に衝突
    - ボールをy方向に反転する
    - ブロックを消す
  - ブロック左側、右側
    - ボールをx方向に反転する
    - ブロックを消す
  - ブロック頂点
    - ボールをx方向とy方向に反転する
    - ブロックを消す

課題06　ボールが衝突したらブロックを消す

- 課題　kadai06
- ボールがブロックに衝突したらブロックを消す
- どうやって？
  - メソッド void remove()
    見えなくする　　visible = false;
  - 修正：メソッド void display()
    ブロックが見える場合(visible == true)　四角を表示する
  - 修正：メソッド　各衝突判定のメソッド
    ブロックが見える場合　かつ　各衝突条件を満たす場合…に修正

kadai06 Blockクラス

```
class Block {

  float x;
  float y;
  float w;
  float h;
  boolean visible;

  Block(float tempX, float tempY) {
    x = tempX;
    y = tempY;
    w = 80;
    h = 20;
    visible = true;
  }

  void display(){
```

ブロックが見える場合
四角を表示する

```
  }

  void remove(){
    visible = false;
  }
```

```
  boolean isHitCorner(Ball b) {
    if(visible &&
    (dist(b.x, b.y, x, y) < b.radius ||
     dist(b.x, b.y, x, y+h) < b.radius ||
     dist(b.x, b.y, x+w, y) < b.radius ||
     dist(b.x, b.y, x+w, y+h) < b.radius)){
      return true;
    } else {
      return false;
    }
  }
  boolean isHitTopSide(Ball b) {
```

ブロックが見える かつ
ブロック上側に衝突する場合
四角を表示する

```
  }
  boolean isHitBottomSide(Ball b) {
```

ブロックが見える かつ
ブロック下側に衝突する場合
四角を表示する

```
  }
```

kadai06　Blockクラス

```
boolean isHitLeftSide(Ball b) {
```

ブロックが見える　かつ
ブロック左側に衝突する場合
四角を表示する

```
}

boolean isHitRightSide(Ball b) {
```

ブロックが見える　かつ
ブロック右側に衝突する場合
四角を表示する

```
}

}
```

kadai06 メインタブ

```
// メインタブ（一部省略）

  if(block.isHitTopSide(ball) || block.isHitBottomSide(ball)){
    ball.flipYdir();
    block.remove();
  }
  if(block.isHitRightSide(ball) || block.isHitLeftSide(ball)){
    ball.flipXdir();
    block.remove();
  }
  if(block.isHitCorner(ball)){
    ball.flipXdir();
    ball.flipYdir();
    block.remove();
  }

  ball.display();
  bar.display();
  block.display();
}
```

修正